thread, or the time period specified by `relative_time` has elapsed or the thread is woken spuriously. The `lock` object is locked again before the call to `wait_for()` returns.

Returns

`std::cv_status::no_timeout` if the thread was woken by a call to `notify_one()` or `notify_all()` or a spurious wakeup, `std::cv_status::timeout` otherwise.

Throws

`std::system_error` if the effects can't be achieved. If the `lock` object is unlocked during the call to `wait_for()`, it's locked again on exit, even if the function exits via an exception.

Note

The spurious wakeups mean that a thread calling `wait_for()` may wake even though no thread has called `notify_one()` or `notify_all()`. It's therefore recommended that the overload of `wait_for()` that takes a predicate is used in preference where possible. Otherwise, it's recommended that `wait_for()` be called in a loop that tests the predicate associated with the condition variable. Care must be taken when doing this to ensure that the timeout is still valid; `wait_until()` may be more appropriate in many circumstances. The thread may be blocked for longer than the specified duration. Where possible, the elapsed time is determined by a steady clock.

Synchronization

Calls to `notify_one(), notify_all(), wait(), wait_for()`, and `wait_until()` on a single